EV Analytics and Visualization Project Documentation

1. PROJECT TITLE

EV Analytics and Visualization System Using Tableau

2. PROJECT DESCRIPTION

System Overview

This project is a lightweight EV analytics system that generates Tableau-ready datasets and supports dashboard creation for EV charging analysis, battery monitoring, and model comparison. The implementation uses a Python-based data pipeline and CSV outputs that can be imported directly into Tableau.

Problem Domain

Electric vehicle adoption is increasing across cities, fleets, and personal transportation. This creates a need to understand charging demand, battery health, driving efficiency, and model-level cost and sustainability metrics. Raw EV data is difficult to interpret without analytics and visualization.

Core Technology Used

- Python 3 for data generation and preprocessing
- CSV files as the exchange format
- Tableau Desktop or Tableau Public for dashboard creation
- Markdown documentation for project guidance

Key Capabilities

- Generates synthetic EV charging session data
- Generates battery performance and fleet summary data
- Generates EV model comparison data
- Produces pre-aggregated CSV files for faster Tableau analysis
- Supports multiple stakeholder-specific dashboard scenarios

Target Users

- City energy planners
- EV fleet managers
- EV consumers
- Students and researchers learning data analytics

Real-world Relevance

The project reflects real EV analytics needs such as grid planning, charging infrastructure expansion, predictive maintenance, operational efficiency, and sustainable consumer decision-making.

3. APPLICATION SCENARIOS / USE CASES

Enterprise Usage

A municipal planning team can use the charging analytics dashboard to identify peak charging hours, high-demand neighborhoods, and charger-type usage before expanding public EV infrastructure.

Industrial Usage

An EV taxi or delivery fleet can use the battery performance dashboard to compare predicted range with actual range, monitor battery degradation, and identify vehicles that need maintenance.

End-user Usage

A buyer comparing EV models can use the model efficiency dashboard to evaluate price, range, energy efficiency, charging cost, and carbon savings before making a purchase.

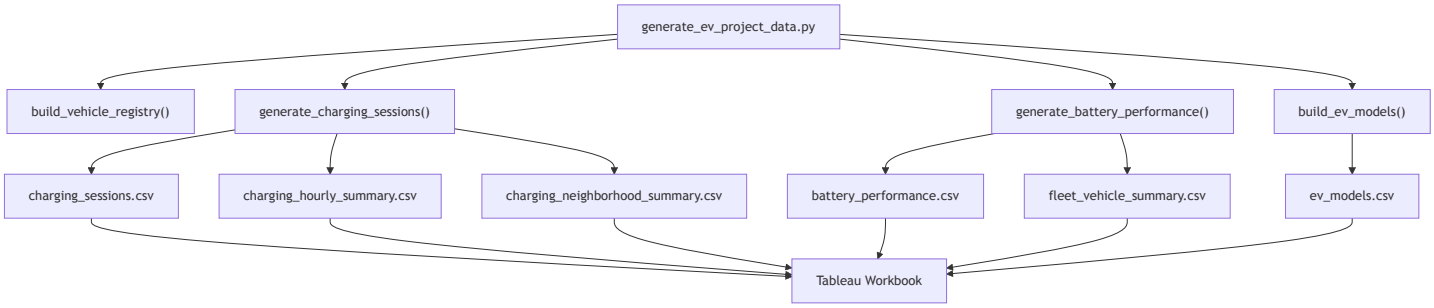
Educational Usage

Students can use the generated data in Tableau labs to learn data cleaning, KPI design, dashboard building, and EV analytics concepts without needing access to private real-world datasets.

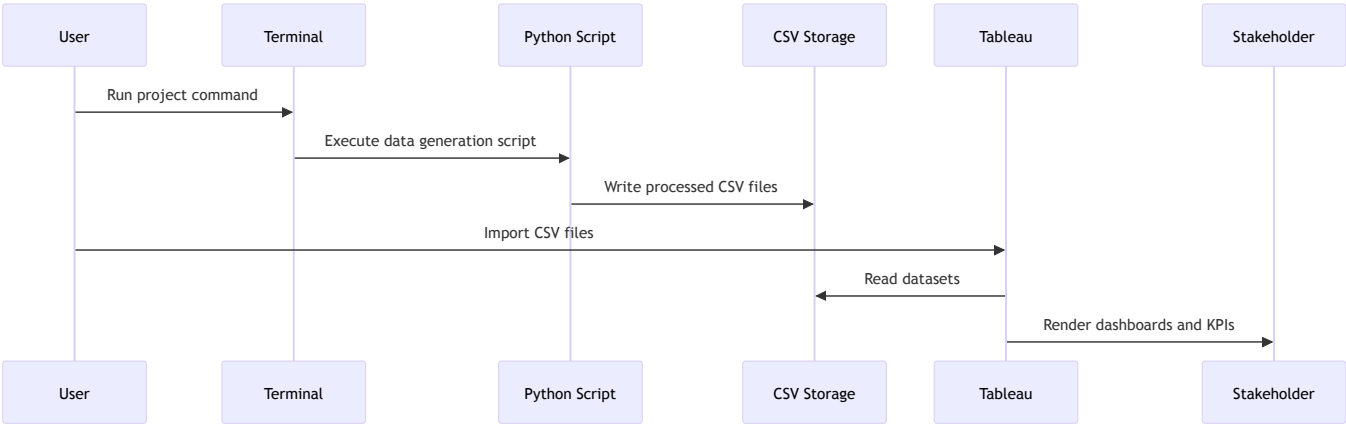
4. TECHNICAL ARCHITECTURE OVERVIEW

High-level Architecture Diagram

Component Interaction Diagram



Data / Request Flow



Frontend Layer

The frontend layer is Tableau, where dashboards, charts, filters, and KPI cards are created for end users.

Backend Layer

The backend layer is the Python data generation script in `scripts/generate_ev_project_data.py` . It simulates EV operational data and builds summary datasets for analysis.

Database Layer

There is no relational database in the current implementation. The project uses flat CSV files stored under `data/processed/` as the data layer.

External APIs / Services

No external API is required in the current version. Tableau Public, Tableau Cloud, or Tableau Server may be used later for sharing dashboards.

Deployment Environment

The current project is intended to run on a local development machine. Dashboards can optionally be published to Tableau Public or Tableau Server for wider access.

5. PREREQUISITES

5.1 Software Requirements

- Development tools: terminal, Git, and any text editor
- IDE: VS Code, PyCharm, or equivalent
- Runtime environment: Python 3.10 or later
- Visualization software: Tableau Desktop or Tableau Public
- Cloud services: not required in the current version

5.2 Libraries / Frameworks / Dependencies

- Python standard library modules: `csv`, `random`, `collections`, `dataclasses`, `datetime`, `pathlib`, `subprocess`, `unittest`
- No mandatory third-party Python packages
- Tableau for visualization

5.3 Hardware Requirements (If Applicable)

- Minimum 4 GB RAM, 8 GB recommended
- Dual-core processor or better
- 200 MB free storage for project files and generated data
- No external devices or sensors required in the current version

6. PRIOR KNOWLEDGE REQUIRED

- Basic Python programming
- Basic Tableau usage
- Dataset and CSV handling fundamentals
- Data visualization and KPI concepts
- Basic understanding of electric vehicles and battery terms

7. PROJECT OBJECTIVES

Technical Objectives

- Build a reusable Python data pipeline for EV analytics datasets
- Create dashboard-ready data for charging, battery, and model comparison analysis
- Document the project in an implementation-oriented format

Performance Objectives

- Generate datasets quickly on a standard local machine
- Keep outputs lightweight for fast Tableau import
- Provide summary files to reduce dashboard computation overhead

Deployment Objectives

- Enable local execution without external services
- Support easy import into Tableau Desktop or Tableau Public
- Prepare the project for later publishing on Tableau Cloud or Tableau Server

Learning Outcomes

- Understand EV analytics problem framing
- Learn how raw data becomes dashboard-ready datasets
- Learn how to design analytics outputs for different user groups

8. SYSTEM WORKFLOW

1. User interaction: the user runs the generator script from the terminal.
2. Input handling: the script loads predefined EV models, neighborhoods, charger types, and simulation rules.
3. Processing logic: the script generates charging sessions, battery records, and EV model comparison data.
4. Core system execution: summary tables are computed for hourly charging demand, neighborhood demand, and fleet performance.
5. Output generation: all generated datasets are written as CSV files into `data/processed/` .
6. Response delivery: Tableau reads the CSV files and displays dashboards for planners, fleet managers, and consumers.

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition

Stakeholders need an understandable way to explore EV data across charging usage, battery performance, and model efficiency. The challenge is to convert raw or simulated EV data into clear dashboards that support operational and strategic decisions.

9.2 Functional Requirements

- Generate EV charging session data
- Generate EV battery performance data
- Generate EV model comparison data
- Produce summary datasets for Tableau dashboards
- Provide documentation for execution and dashboard mapping
- Support three analytical scenarios from one project workspace

9.3 Non-Functional Requirements

- Fast execution on local machines
- Simple setup with minimal dependencies
- Reproducible outputs
- Readable code and documentation
- Privacy-safe sample data

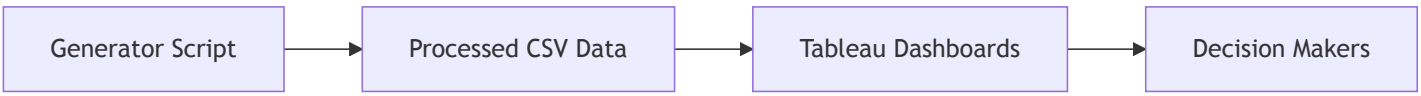
9.4 System Design Decisions

- Synthetic data is used to avoid privacy and availability issues
- CSV is used because Tableau imports it directly
- Pre-aggregated summary files are generated for faster dashboard design
- A single script is used to keep the pipeline simple and portable
- The project is local-first to reduce setup complexity

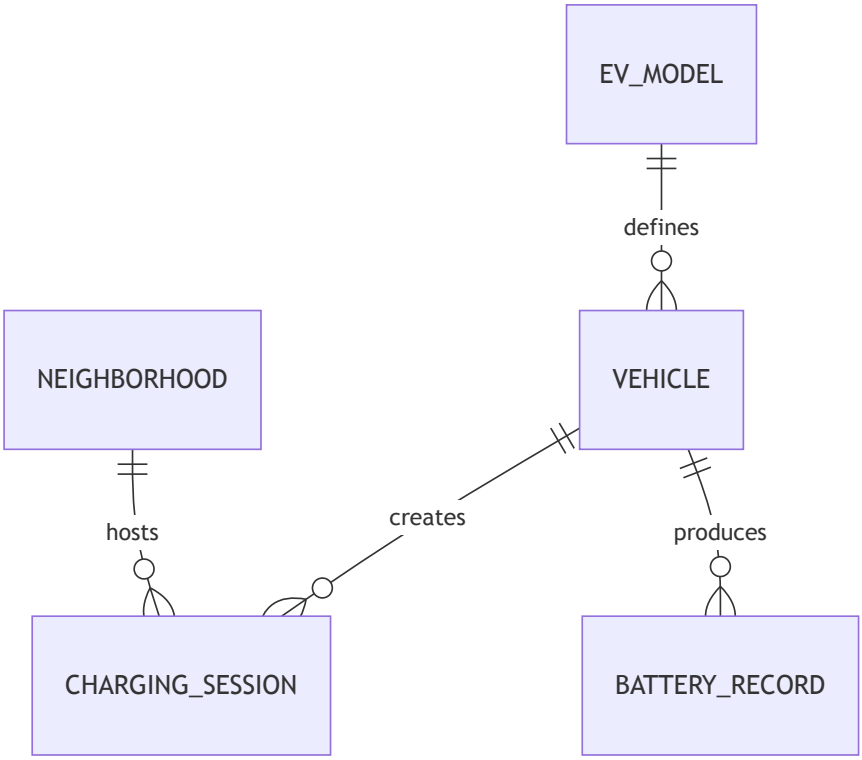
9.5 Technology Stack Selection Justification

- Python was selected because it is simple, portable, and effective for data generation
- The standard library was selected to avoid installation overhead
- Tableau was selected because it is strong for interactive analytics and dashboarding
- CSV was selected because it is easy to inspect, version, and import

Architecture Diagram



ER Diagram (Logical Data Model)



API Structure

No API layer exists in the current implementation. The system is file-based and locally executed.

10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

10.1 Development Environment Setup

Recommended steps:

```
cd /Users/charitsinghchauhan/Documents/code/da
python3 --version
```

Optional virtual environment setup:

```
python3 -m venv .venv
source .venv/bin/activate
```

10.2 Dependency Installation

No third-party dependency installation is required for the current project.

Generate the project outputs with:

```
python3 scripts/generate_ev_project_data.py
```

Run the validation tests with:

```
python3 -m unittest discover -s tests -v
```

10.3 Project Structure Creation

The main working structure for this project is:

```
project_root/
├── data/
│   └── processed/
├── docs/
├── scripts/
├── tests/
├── .gitignore
├── README.md
└── requirements.txt
```

10.4 Configuration Setup

- No `.env` file is required
- Output files are written to `data/processed/`
- The generator uses a fixed random seed for reproducible datasets
- Tableau reads the generated CSV files directly

Folder Structure

- `data/processed/` : generated CSV datasets
- `docs/` : project report and dashboard guide
- `scripts/` : Python script for dataset generation
- `tests/` : smoke tests for output validation
- `README.md` : quick-start instructions
- `requirements.txt` : dependency note for the project

Setup Commands

```
cd /Users/charitsinghchauhan/Documents/code/da
python3 scripts/generate_ev_project_data.py
python3 -m unittest discover -s tests -v
```

Configuration Explanation

The project is intentionally configuration-light. Data size, date range, EV model definitions, neighborhoods, and charger types are defined inside the generator script for simplicity.

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

11.1 Feature 1 - Charging Analytics Data Pipeline

Description

This feature generates detailed EV charging session data and summary views required for the charging pattern dashboard.

Code Explanation

- `generate_charging_sessions()` creates timestamped session records with charger type, duration, energy consumed, and neighborhood
- `summarize_charging_by_hour()` produces hourly session counts and average charging metrics
- `summarize_charging_by_neighborhood()` produces neighborhood-level charging demand summaries

Screenshots / Outputs

Generated files:

- `data/processed/charging_sessions.csv`
- `data/processed/charging_hourly_summary.csv`
- `data/processed/charging_neighborhood_summary.csv`

11.2 Feature 2 - Battery Performance Monitoring Pipeline

Description

This feature generates fleet battery performance records and a vehicle-level summary for range and health analysis.

Code Explanation

- `generate_battery_performance()` simulates battery health, state of charge, temperature, cycle count, predicted range, and actual range
- `summarize_fleet()` aggregates battery performance by vehicle for fleet management dashboards

Screenshots / Outputs

Generated files:

- `data/processed/battery_performance.csv`
- `data/processed/fleet_vehicle_summary.csv`

11.3 Feature 3 - EV Model Comparison Dataset

Description

This feature provides model-level EV data for consumer comparison dashboards.

Code Explanation

- `build_ev_models()` creates a compact model comparison dataset with battery size, range, price, efficiency, cost per kilometer, and annual savings

Screenshots / Outputs

Generated file:

- `data/processed/ev_models.csv`

Database Schema

The current implementation uses flat CSV datasets rather than a relational database. The logical entities are EV models, vehicles, charging sessions, battery records, and neighborhoods.

API Endpoints

No API endpoints exist in the current implementation.

Model Architecture

The system uses a rule-based synthetic data model rather than a machine learning model. Weighted probabilities and simple degradation formulas are used to create realistic patterns.

Business Logic Layer

The business logic is contained in the generator script and includes:

- weighted charger and neighborhood selection
- session duration and energy simulation
- battery degradation and range estimation
- summary aggregation for Tableau consumption

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

12.1 Component Integration

The project integrates the Python data generator, processed CSV files, project documentation, and Tableau dashboards into a single workflow. The script generates the data, the `data/processed/` folder stores the outputs, and Tableau consumes those outputs for visualization.

12.2 Performance Optimization

- Pre-aggregated summary CSV files reduce Tableau calculation load
- The generator uses efficient standard library operations
- Dataset sizes are intentionally compact for quick local iteration

12.3 Security Enhancements

- Synthetic data avoids exposure of private vehicle or customer information
- No API keys, credentials, or cloud secrets are required
- Local file-based execution reduces attack surface

12.4 Error Handling & Validation

- Output directories are created automatically if missing

- CSV column schemas are defined explicitly through field names
- Reproducible generation helps detect unintended changes
- Smoke tests validate file creation, row counts, and core columns

13. MILESTONE 5: TESTING & VALIDATION

13.1 Test Cases

Test Case ID	Input	Expected Output	Status
TC-01	Run `python3 scripts/generate_ev_project_data.py`	All six CSV files are created in `data/processed/`	Passed
TC-02	Inspect `charging_sessions.csv`	1500 charging session records are present	Passed
TC-03	Inspect `battery_performance.csv`	900 battery performance records are present	Passed
TC-04	Inspect `ev_models.csv`	8 EV model records are present	Passed
TC-05	Run `python3 -m unittest discover -s tests -v`	Smoke tests pass successfully	Passed

13.2 Unit Testing

Unit-level validation is provided through `tests/test_generated_data.py` , which checks file existence, row counts, and required columns.

13.3 Integration Testing

Integration testing verifies that:

- the generator script writes files successfully
- the produced CSV files match the expected structure
- Tableau can use the files as direct input sources

13.4 User Acceptance Testing

Formal user acceptance testing with real stakeholders is not yet completed. Current acceptance is scenario-based and validates whether the datasets support the planner, fleet, consumer, and educational use cases.

13.5 Performance Metrics

For the current analytics pipeline:

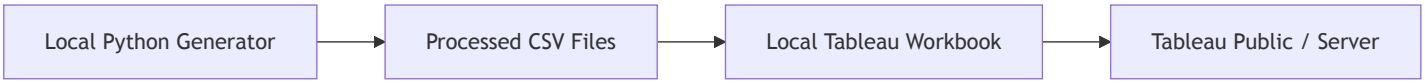
- Dataset generation time observed locally: approximately `0.1621` seconds
- Charging session records generated: `1500`
- Battery performance records generated: `900`
- EV model records generated: `8`

For this project, AI/ML metrics such as accuracy, precision, recall, and F1 score are not applicable. Web response-time metrics are also not applicable because the system is not a live web application.

14. DEPLOYMENT

14.1 Deployment Architecture

The project is deployed locally for data generation and dashboard authoring. For sharing, the final Tableau workbook can be published to Tableau Public, Tableau Cloud, or Tableau Server.



14.2 Hosting Platform

- Local machine for development
- Tableau Desktop for workbook creation
- Tableau Public, Tableau Cloud, or Tableau Server for dashboard publishing

14.3 Deployment Steps

1. Run the Python generator script.
2. Verify the CSV outputs.
3. Import the CSV files into Tableau.
4. Build the dashboards.
5. Publish the final workbook if external sharing is required.

14.4 Production Considerations

- Replace synthetic data with real EV data sources
- Add scheduled refresh workflows
- Introduce access control for shared dashboards
- Monitor data quality and version dashboard files

15. PROJECT STRUCTURE

The main working structure of the current implementation is shown below. This project is a lightweight analytics workflow rather than a web application:

```
project_root/
│
├── data/
│   └── processed/
├── docs/
├── scripts/
├── tests/
├── .gitignore
├── README.md
└── requirements.txt
```

Folder Explanation

- `data/processed/` : stores Tableau-ready CSV outputs

- `docs/` : contains the detailed project report and Tableau guide
- `scripts/` : contains the Python generator script
- `tests/` : contains smoke tests for output validation
- `README.md` : contains run instructions and project overview
- `requirements.txt` : documents the dependency situation for the project

16. RESULTS

System Output

The system successfully generates six CSV datasets for the three dashboard scenarios.

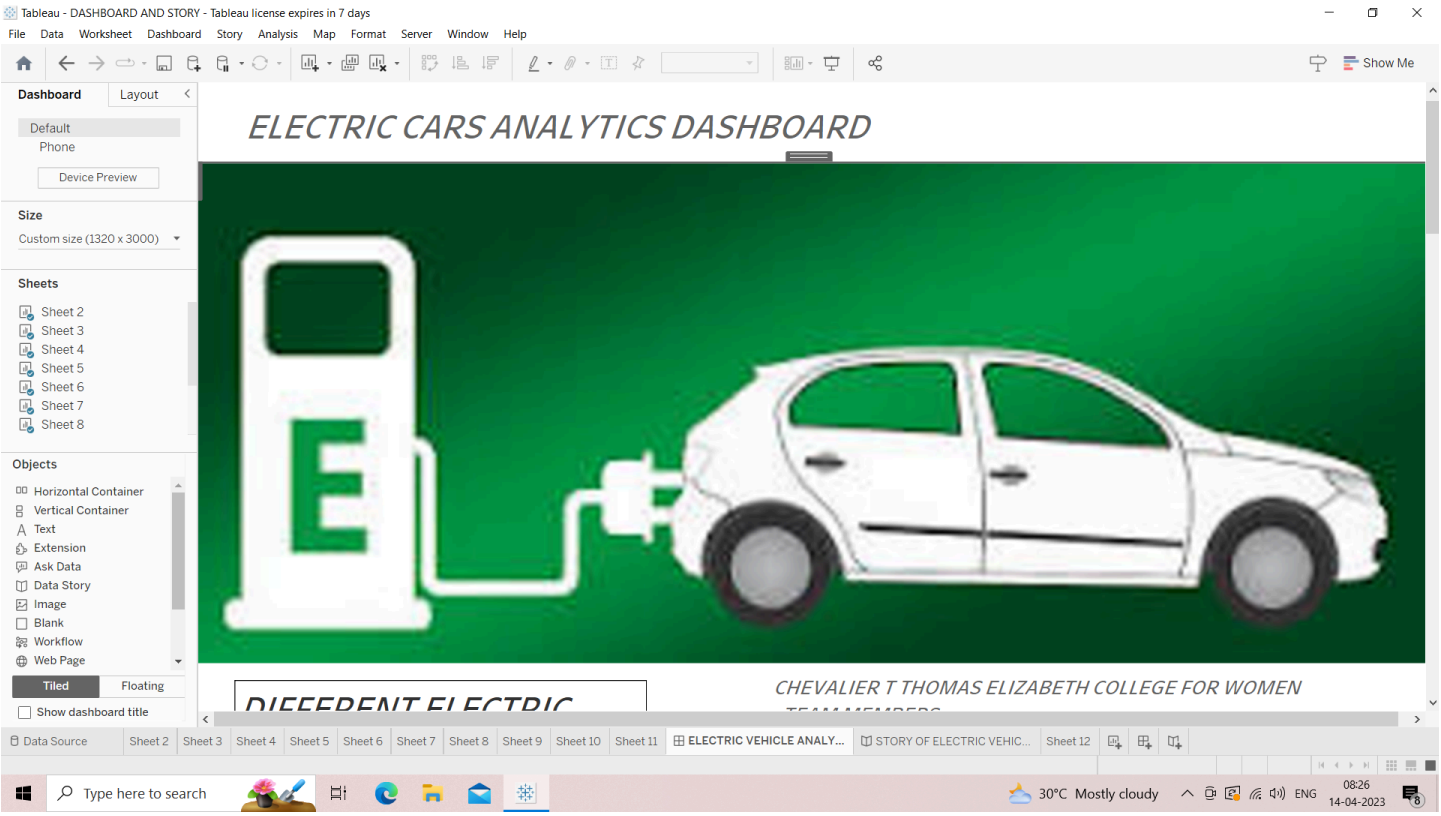
Performance Evaluation

The generator completed in approximately `0.1621` seconds during local validation, showing that the system is lightweight and suitable for quick iterative development.

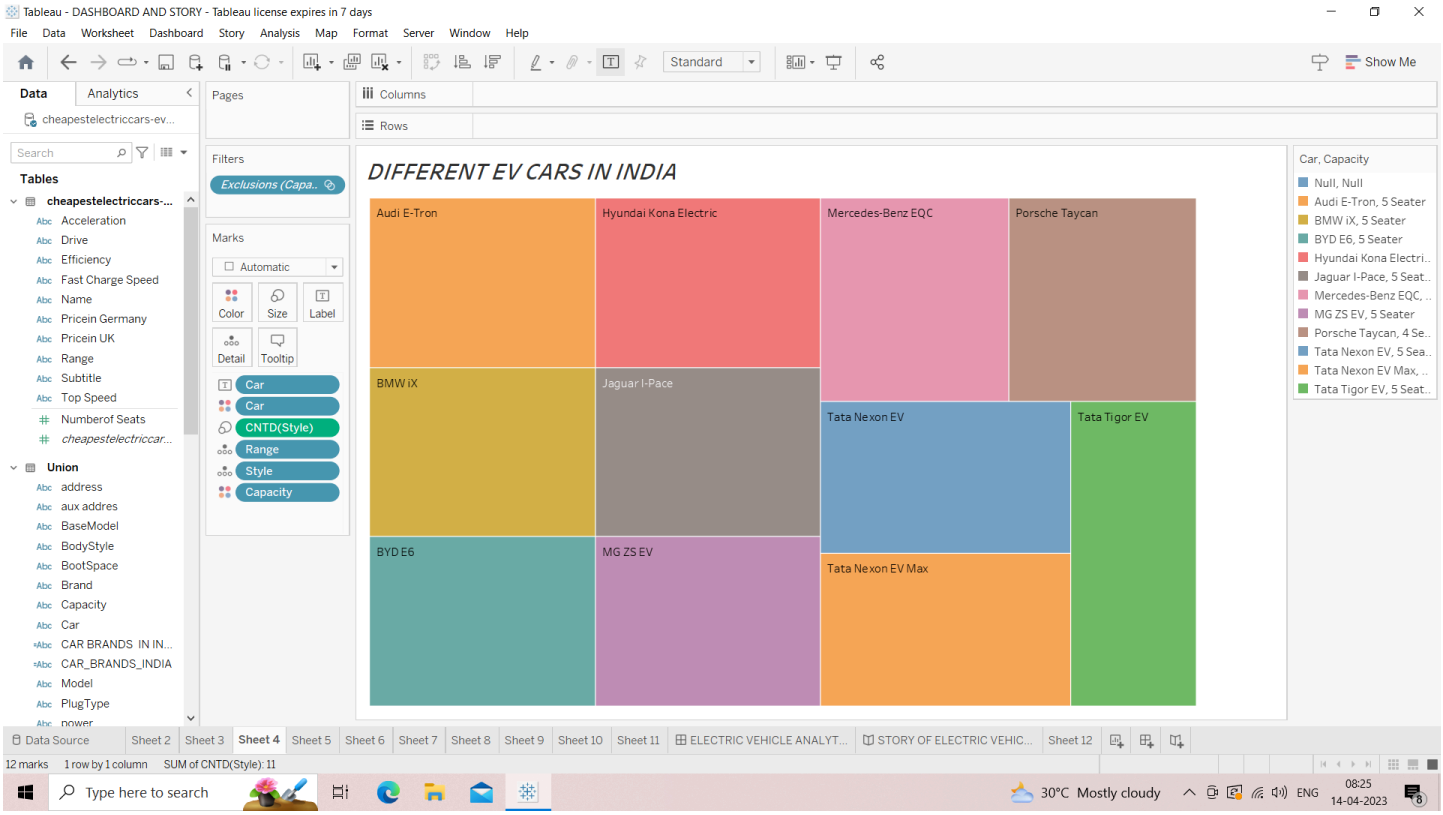
Screenshots

The following screenshots are taken from the existing Tableau project stored in `project/visualization-tool-for-electric-vehicle-charge-and-range-analysis/output screenshorts/Screenshots`.

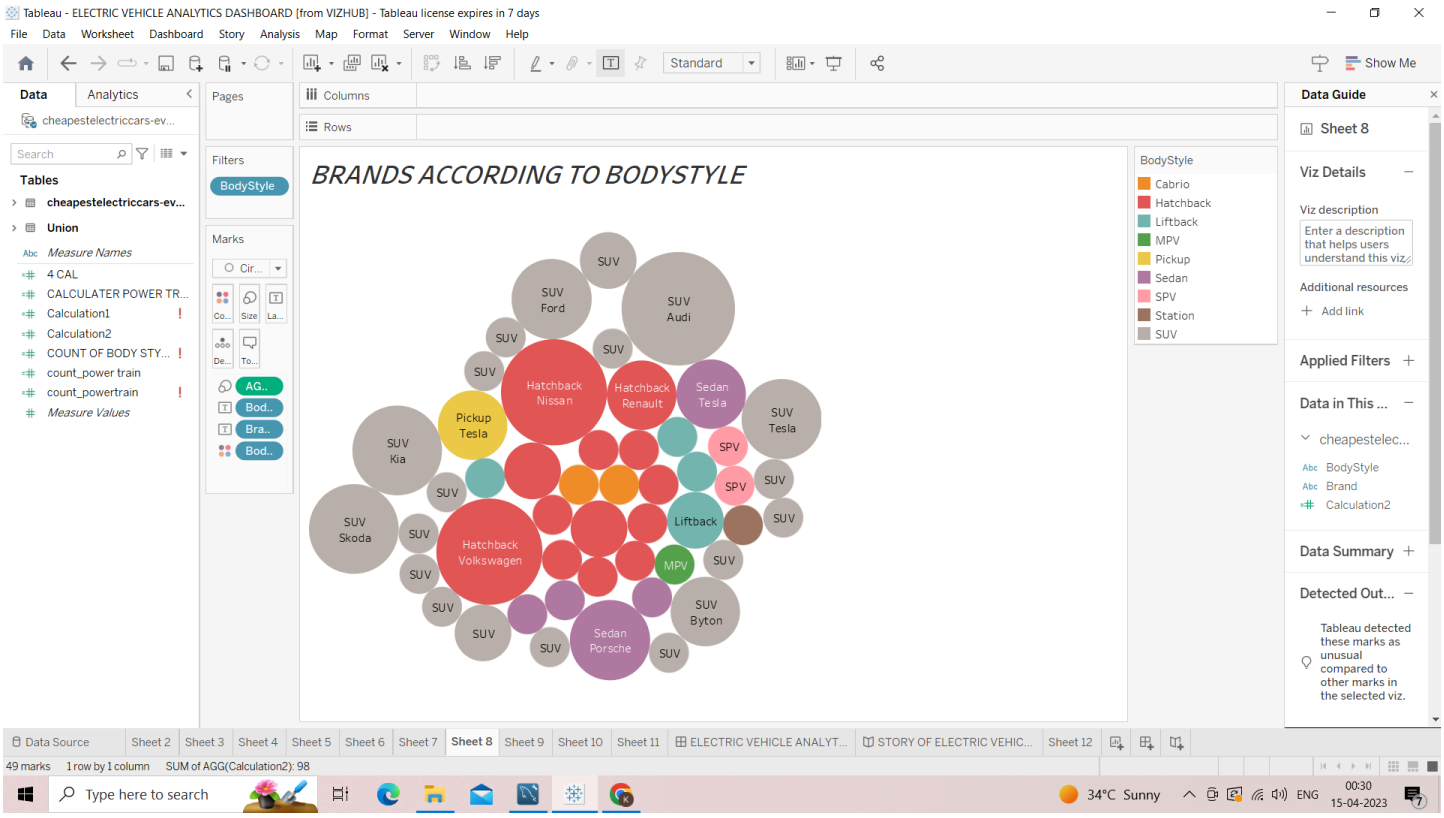
Dashboard Overview



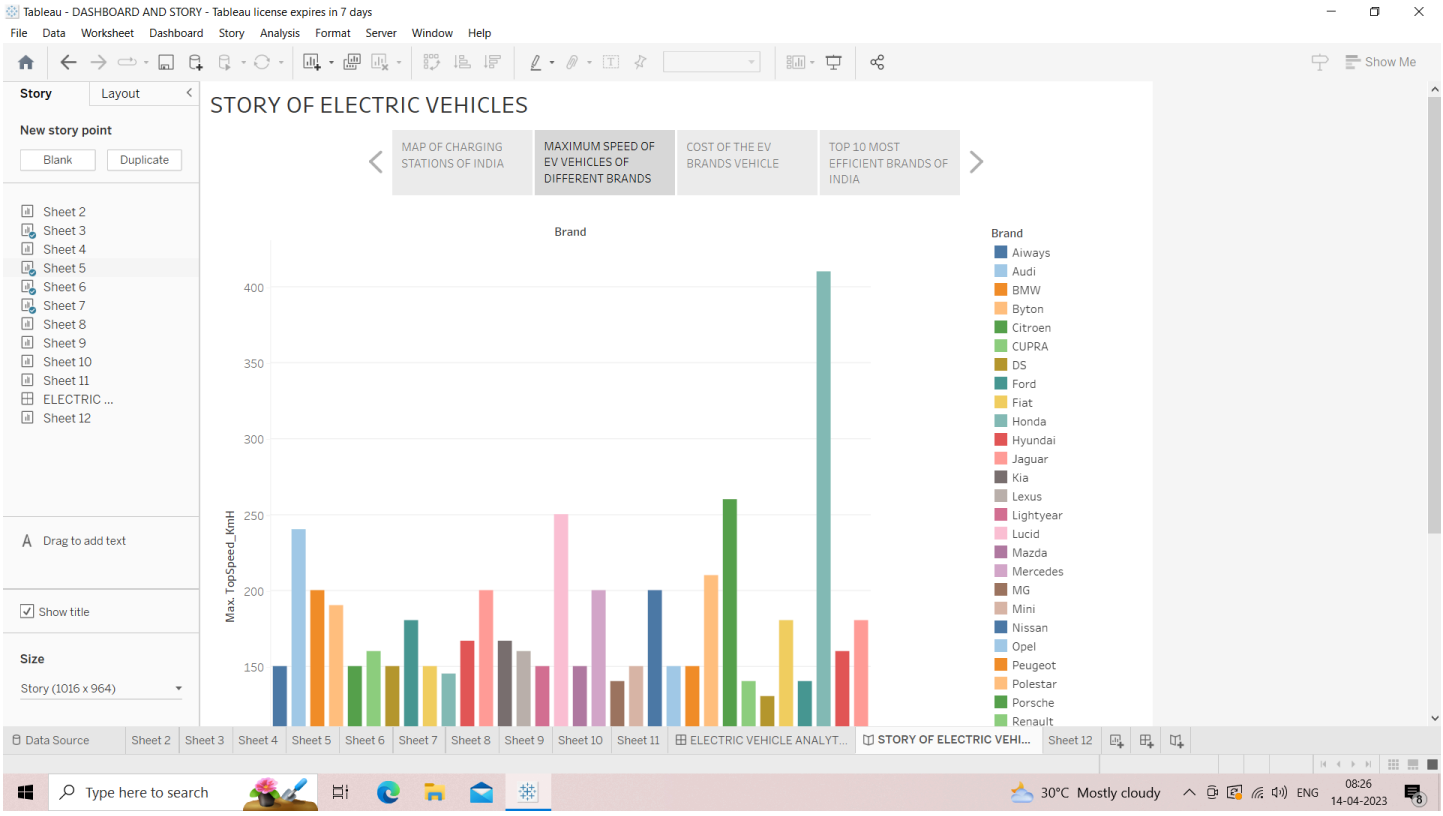
EV Model Comparison Treemap



Price Comparison View



Story View



Benchmark Results

Output File	Result
`charging_sessions.csv`	1500 rows
`charging_hourly_summary.csv`	24 rows
`charging_neighborhood_summary.csv`	8 rows
`battery_performance.csv`	900 rows
`fleet_vehicle_summary.csv`	30 rows
`ev_models.csv`	8 rows

17. ADVANTAGES & LIMITATIONS

Advantages

- Simple setup with no mandatory third-party Python dependencies
- Tableau-ready outputs generated in a single command
- Supports multiple real-world EV analytics scenarios
- Uses synthetic data, which is safe for academic and demo use
- Easy to extend with new metrics and additional datasets

Limitations

- No live data ingestion
- No production backend or API layer

- No relational database
- No Tableau workbook file committed yet
- User acceptance testing is still limited to scenario validation

18. FUTURE ENHANCEMENTS

- Add scalable storage such as PostgreSQL or Parquet
- Add new features such as charging demand forecasting and battery anomaly detection
- Integrate cloud publishing and scheduled refresh workflows
- Provide mobile-friendly dashboard layouts
- Automate dataset generation and validation through CI or scheduled jobs

19. CONCLUSION

This project implements a practical EV analytics starter system using Python and Tableau. It generates relevant datasets for charging analysis, battery monitoring, and EV model comparison with minimal setup. The project demonstrates clear technical value for learning, prototyping, and early-stage dashboard development, while also leaving room for future expansion into live data and cloud deployment.

20. APPENDIX

Source Code

- `scripts/generate_ev_project_data.py`
- `tests/test_generated_data.py`

Configuration Files

- `.gitignore`
- `requirements.txt`

Dataset Details

- `charging_sessions.csv` : detailed charging session records
- `charging_hourly_summary.csv` : hourly aggregate charging metrics
- `charging_neighborhood_summary.csv` : neighborhood-level charging demand
- `battery_performance.csv` : vehicle battery performance records
- `fleet_vehicle_summary.csv` : vehicle-level battery performance summary
- `ev_models.csv` : EV model comparison dataset